

Model Compression and Efficiency Optimisation for Edge Deployment

Diploma Seminar 1 — Introduction and Literature Review

Hubert Jaczyński

Warsaw, 2026

Warsaw University of Technology

Supervisor: PhD. Maria Ganzha

Agenda

1. Motivation: Why object detection on edge devices is difficult
2. Research goals + questions
3. Background: CNNs vs Transformers (vision)
4. Efficiency toolbox:
 - Quantisation (PTQ vs QAT)
 - Pruning (structured vs unstructured)
 - Knowledge distillation
 - Hardware-aware NAS / design
5. Future work
6. Summary

What are edge devices?

- **Edge device** = the model runs *close to where the data is produced*, instead of on a cloud server.
- Compared to cloud inference, edge inference typically has:
 - **tight compute and memory limits** (smaller accelerators, less RAM),
 - **real-time requirements** (low latency / steady FPS),
 - **practical constraints** like battery life, privacy, and unreliable connectivity.
- In short: we need models that are not only accurate, but also **efficient at inference time**.

Variants of edge devices

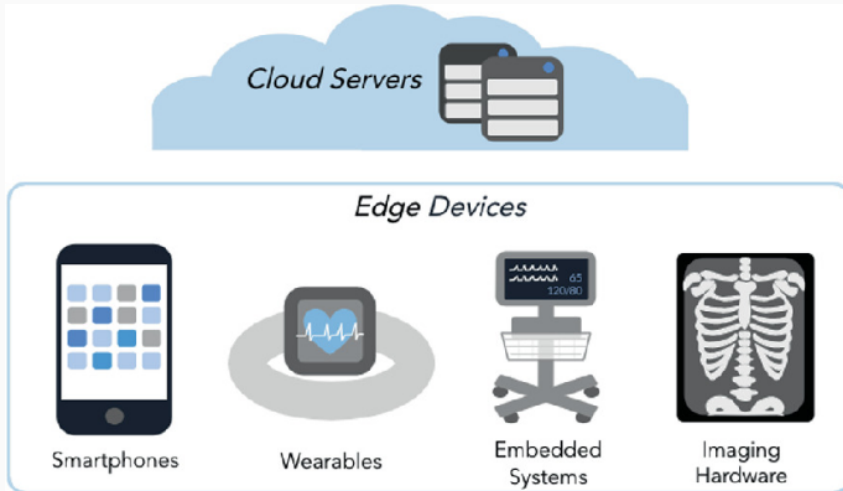


Figure: Variants of edge devices (reproduced from [1]).

Motivation: accuracy is not the only constraint

- High benchmark accuracy does not automatically translate to a usable model in production.
- In edge/real-time settings (mobile, cameras, robots), inference is constrained by:
 - limited throughput for dense ops (e.g., convolutions / attention) on the available accelerator,
 - memory pressure from weights and feature maps (and the cost of moving them),
 - power limits that reduce sustained performance.
- The goal is **real-time object detection** with competitive accuracy under these constraints.
- This creates a practical trade-off: **accuracy** vs **latency** vs **model size** (and ideally energy).

Related survey viewpoint: [2, 3].

Edge deployment: what “real-time” means

- **Real-time** means the detector can process a live stream *without visible lag* (typically **15–30 FPS**, depending on the application).
- The end-to-end latency is not only the neural network. A full inference pipeline includes:
 - input preprocessing (decode / resize / normalise),
 - model forward pass (backbone + neck + head),
 - postprocessing (box decoding, NMS),
 - data movement and I/O (camera capture, memory copies).
- Important practical point: **FLOPs/MACs are only a rough proxy** — actual latency depends on operator support, kernel efficiency, and memory bandwidth on the target device.

Aim of the work

- The focus is **real-time object detection on edge devices**, where compute and memory are limited.
- I compare efficiency methods that reduce **inference cost** (compute, memory, bandwidth) while keeping detection quality as high as possible.
- The main evaluation criteria are **accuracy** (detection metric), **latency** (ms / FPS), and **model size**; **energy** is included when measurement is feasible.

Background: architectures

Convolutional Neural Networks (CNNs): short introduction

- A **CNN** is a neural network designed for images, where the main operation is the **convolution**: a small learnable filter is applied across the image to detect local patterns.
- Stacking convolution layers builds a hierarchy of features: **edges/texture** → **parts** → **objects**.
- For object detection, CNNs are typically used as a **backbone** that produces feature maps, which are then processed by a detection head (classification + box regression).

CNNs in vision: why they work well on edge

- CNNs are well-suited for images because they look at **small local regions** at a time and reuse the **same filter** across the whole image. This makes them efficient and practical to run.
- A rough compute proxy for a convolution layer is the number of multiply-accumulate operations:

$$\text{MACs} \approx H'W' \cdot C_{\text{out}} \cdot (k^2 C_{\text{in}}).$$

- In practice, the **early layers** (where feature maps are still large) often take most of the time, and **moving data in memory** can be a bottleneck even if the MAC count looks reasonable.

See efficiency discussion: [2].

CNN building blocks that matter for efficiency

- **Residual connections** (ResNet-style) add skip paths that stabilise training, which enables deeper backbones without vanishing gradients.
- **Bottleneck blocks** use 1×1 convolutions to reduce/expand channels around an expensive 3×3 convolution, lowering compute while keeping representational power.
- **Downsampling choices** (max-pooling vs strided convolutions) achieve similar spatial reduction, but differ in learnability, compute cost, and memory-access patterns.
- These blocks define natural **units for optimisation** later: pruning channels/blocks, or quantising weights/activations within them.

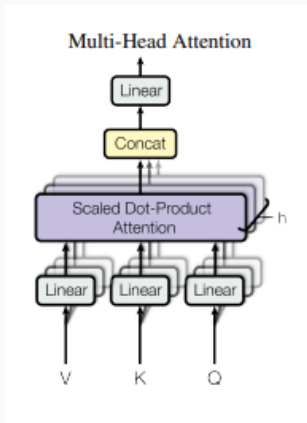
Background: [4].

Transformers: what they are (high level)

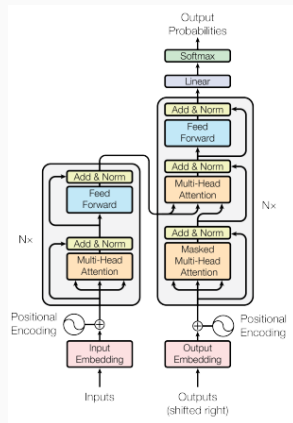
- A **Transformer** is a neural network that processes a **sequence of tokens** and builds contextual representations for each token.
- The key operation is **self-attention**: each token combines information from other tokens, allowing the model to capture long-range dependencies.
- In practice, Transformers alternate:
 - **(Multi-head) self-attention** for information mixing,
 - a **feed-forward network (MLP)** applied per token.
- Unlike CNNs, Transformers do not assume locality by default, which can help modeling global context, but often increases computation.

Canonical reference: [5].

Transformer architecture visualisation



Multi-head self-attention (adapted from [5]).



Transformer encoder-decoder overview (adapted from [5]).

Vision Transformers (ViT): applying Transformers to images

- ViT represents an image as a **sequence of patch tokens**: split into $P \times P$ patches and project each patch to an embedding.
- The number of tokens grows with resolution:

$$n = \frac{HW}{P^2}.$$

- After adding positional information, the patch tokens are processed by standard Transformer layers (attention + MLP) to produce image features.
- For detection, ViT features can be used as a backbone (often combined with CNN-like components).

ViT: [6].

Making Transformers efficient for vision (why and how)

- Main cost driver: global attention compares all token pairs $\Rightarrow$ **quadratic cost** $O(n^2)$ in the number of tokens, which grows quickly with image resolution.
- Common efficiency ideas in vision transformers:
 - **Local / windowed attention** (attend within neighborhoods instead of globally),
 - **Hierarchical representations** (downsample tokens across stages, similar to CNN feature pyramids),
 - **Efficient attention variants** (approximate/sparse attention, linear-time forms),
 - **Hybrid models** (CNN stem or mixed blocks) to keep locality where it helps.
- Practical takeaway for edge: CNNs are often simpler to deploy efficiently, while ViTs may need these design choices (plus quantisation/pruning) to meet real-time constraints.

Efficiency toolbox

Numeric precision in inference (FP32, FP16, INT8)

- Neural networks store **weights** and compute **activations** using finite-precision numbers.
- **FP32** (32-bit float) is the default in training: high precision, but larger memory and slower inference.
- **FP16** (16-bit float) is common for faster inference/training on GPUs: similar accuracy in many cases.
- **INT8** (8-bit integer) is widely used for edge inference: much smaller tensors and potentially faster kernels.
- Lower precision saves memory/bandwidth, but can introduce **quantisation error** that may reduce accuracy.

Quantisation: idea and why it helps

- **Quantisation** replaces FP32 computations with lower precision (typically **FP16** or **INT8**) for weights and/or activations.
- A common approach is an affine mapping with a scale s and zero-point z :

$$x_q = \text{clip} \left(\left\lfloor \frac{x}{s} \right\rfloor + z, q_{\min}, q_{\max} \right), \quad \hat{x} = s(x_q - z).$$

- In practice, the main benefits are:
 - **smaller model** and lower memory bandwidth (especially for activations),
 - **faster inference** when the runtime/device has efficient INT8 kernels.

Surveys: [7, 2].

PTQ vs QAT: two ways to get an INT8 model

Post-training quantisation (PTQ)

- Start from a trained FP32 model and quantise it after training.
- Use a small calibration set to estimate activation ranges.
- Very fast to apply, but accuracy can drop (often more noticeable in detection).

Reference survey: [7].

Quantisation-aware training (QAT)

- Train with **fake quantisation** so the model learns to be robust to INT8 rounding/clipping.
- Typically achieves higher INT8 accuracy than PTQ.
- Costs additional training effort and careful tuning.

Pruning: remove redundancy in the network

- **Pruning** removes parameters that contribute little to the final prediction, aiming to keep accuracy while reducing inference cost.
- A simple view is applying a binary mask m to parameters:

$$\theta' = m \odot \theta.$$

- Two common types (with different practical outcomes):
 - **Unstructured pruning:** removes individual weights $\Rightarrow$ high sparsity, but the computation becomes irregular.
 - **Structured pruning:** removes whole channels/filters/blocks $\Rightarrow$ a smaller *dense* model that is easier to accelerate.
- For real-time edge inference, **structured pruning is often more useful**, because it reliably reduces tensor sizes and translates to lower latency.

Classic: [8]; survey: [3].

Knowledge distillation: teacher $\rightarrow$ student

- **Goal:** train a smaller *student* model to match a stronger *teacher* while keeping good accuracy.
- The student learns from two sources:
 - **hard targets** (ground-truth labels),
 - **soft targets** (teacher predictions), which contain extra information about class similarities.
- Soft targets are usually produced with temperature τ :

$$p_T(\tau) = \text{softmax}\left(\frac{z_T}{\tau}\right), \quad p_S(\tau) = \text{softmax}\left(\frac{z_S}{\tau}\right).$$

- Training loss typically combines the standard supervised loss with a **distillation loss** (often KL divergence between p_T and p_S).

Distillation: [9].

Distillation in object detection: what changes

- Object detection is not only classification: the output is **structured** (class scores + box regression, sometimes dense maps).
- Therefore, distillation can be applied at multiple levels:
 - **classification logits/scores** (confidence distribution),
 - **box regression outputs** (localisation),
 - **intermediate features** (backbone/neck representations),
 - **dense predictions** (e.g., heatmaps or feature pyramids, depending on the detector).
- In an efficiency pipeline, distillation is often a **recovery step**: after pruning or INT8 quantisation, it helps the student regain lost accuracy.

Hardware-aware NAS / efficient-by-design models

- Instead of compressing a fixed network, **neural architecture search (NAS)** tries to find an architecture α that is accurate *and* meets a deployment budget:

$$\max_{\alpha \in \mathcal{A}} \text{Acc}(\alpha) \quad \text{s.t.} \quad \text{Cost}(\alpha) \leq C_{\max}.$$

- The key is that **Cost** should reflect the *target runtime/device* (e.g., measured latency, memory footprint, sometimes energy), not only FLOPs.
- In practice (especially with limited time/compute), a common workflow is:
 - start from an efficient baseline architecture,
 - restrict the search/tuning to a small design space (e.g., widths, depths, kernel sizes),
 - optionally tune only parts of the detector (neck/head),
 - use a latency predictor or on-device measurements to guide choices.
- Takeaway: **NAS designs efficiency in**, while pruning/quantisation **shrink an existing model**.

Compression pipelines: why combinations matter

- These techniques improve efficiency in **different ways**, so combining them is often better than using only one method:
 - **Pruning** reduces model capacity and tensor sizes → fewer operations in a smaller dense network.
 - **Quantisation** reduces numerical precision → fewer bits to move/store and faster INT8 kernels (when supported).
 - **Distillation** transfers knowledge from a strong teacher → helps recover accuracy after compression.
- A practical pipeline for real-time detection can look like:
 1. choose an efficient baseline detector,
 2. apply **structured pruning** to meet the latency budget,
 3. apply **QAT** to obtain a robust INT8 model,
 4. **distill** from a stronger teacher to regain lost accuracy.
- Important caveat: the final outcome depends strongly on the **hardware and runtime stack**, so results should be validated on the target setup.

Evaluation

Evaluation protocol for fair comparison

- **Goal:** compare methods fairly under a realistic deployment pipeline, not only in theory.
- Keep the setup fixed across experiments:
 - the same input resolution(s), preprocessing, and postprocessing (especially NMS),
 - the same dataset splits, augmentation policy, and training budget.
- Report a consistent set of metrics:
 - **accuracy:** mAP@[.5:.95] (COCO-style) or the dataset-appropriate detection metric,
 - **latency:** end-to-end and model-only time (ms / FPS), with warm-up vs cold-start noted,
 - **model size:** on-disk weights and (if possible) peak runtime memory.
- Benchmark using the **intended inference runtime** (e.g., TFLite / TensorRT / ONNX Runtime), because results can change significantly across stacks.

Deployment stack matters more than people expect

- Even with identical weights, the **observed latency** can differ depending on the deployment stack.
- Differences often come from:
 - operator coverage (e.g., whether INT8 conv/attention is actually supported),
 - kernel implementations and fusions (what gets fused vs executed separately),
 - memory layout and bandwidth effects (activation movement can dominate),
 - backend selection (CPU vs GPU vs NPU / delegate choice).

Energy measurement

- On battery-powered platforms, **energy per inference** can be as important as accuracy or latency.
- Practical measurement options (depending on the device):
 - OS-level counters / onboard sensors (when available),
 - external power meter (e.g., for phones or SBCs),
 - steady-state measurement during repeated inference to estimate average power.
- Report both:
 - **power** in watts (W) during inference,
 - **energy per frame** (J/frame) = average power $\times$ inference latency.

Future work

- This area evolves quickly: new architectures and compression methods appear every year, and results can change as runtimes and hardware improve.
- **Lower-bit quantisation:** beyond INT8, current research increasingly explores **INT4** (and mixed precision) to further reduce memory and bandwidth, with new techniques to stabilise accuracy.
- **New efficient architectures:** vision backbones (CNN/Transformer/hybrid) continue to improve, especially designs targeting latency and memory on specific hardware.
- **Better end-to-end pipelines:** further optimisation of preprocessing, postprocessing (e.g., NMS), and runtime-level fusions can yield speedups without changing the core model.
- These directions are natural candidates for extension in the final thesis and experimental work.

Wrap-up

Key takeaways

- Edge deployment is a **systems problem**: model + hardware + runtime.
- CNNs are still strong for edge; transformers need careful efficiency design.
- Best results often come from **method combinations** (prune + INT8 + distill).
- Measuring on the target stack is essential for answering the research questions.

References

- [1] Areeba Abid et al. **“Optimizing Medical Image Classification Models for Edge Devices”**. In: *Distributed Computing and Artificial Intelligence, Volume 1: 18th International Conference*. Ed. by Kenji Matsui et al. Vol. 327. Lecture Notes in Networks and Systems. Cham: Springer, 2022, pp. 77–87. DOI: 10.1007/978-3-030-86261-9_8. URL: https://link.springer.com/chapter/10.1007/978-3-030-86261-9_8.
- [2] Vivienne Sze et al. **“Efficient processing of deep neural networks: A tutorial and survey”**. In: *arXiv:1703.09039* (2017).

- [3] Yu Cheng et al. **“A survey of model compression and acceleration for deep neural networks”**. In: *arXiv:1710.09282* (2017).
- [4] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. **“Deep learning”**. In: *Nature* 521.7553 (2015), pp. 436–444.
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. **“Attention is all you need”**. In: *NeurIPS*. 2017.
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. **“An image is worth 16x16 words: Transformers for image recognition at scale”**. In: *ICLR*. 2021.

- [7] Amir Gholami, Sehoon Kim, Zhen Dong, et al. **“A survey of quantization methods for efficient neural network inference”**. In: *arXiv:2103.13630* (2021).
- [8] Song Han, Huizi Mao, and William J. Dally. **“Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding”**. In: *ICLR*. 2016.
- [9] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. **“Distilling the knowledge in a neural network”**. In: *arXiv:1503.02531* (2015).
- [10] Han Cai, Ligeng Zhu, and Song Han. **“ProxylessNAS: Direct neural architecture search on target task and hardware”**. In: *ICLR*. 2019.

Thank you for your attention!